Engineering platform strategy: migration, operating model, metrics (12 months)

1. Situation

The company runs 5 Azure DevOps organizations with about 1,000 repositories and 1,500 pipelines. Practices are immature. The target is one GitHub Enterprise instance, with Azure Boards kept in one Azure DevOps organization and engineering metrics built in Microsoft Fabric.

Current position: - Repositories are ahead of plan. - Pipeline conversion is behind plan. - Security wants stronger enforcement. - Some legacy products need x86 builds and cannot run on standard runners. - Metric quality is disputed. - Developers report added friction.

2. Recommendation

Re-scope. Keep moving repositories only when their pipelines move with them. Convert pipelines through shared workflows. Move legacy products to a separate track. Define and fix metrics before any target is set from them. Introduce enforcement in stages.

Why not accelerate: repository moves are already ahead, so speeding up adds more repositories whose pipelines are still in Azure DevOps. That is the main source of split work and friction.

Why not pause: a pause leaves half the estate on each platform, with no progress on the legacy products and no reliable measurement. It also stops the metric work that leaders need.

3. Target architecture

Two states must work together for the whole migration, so the design supports both.

Distributed state (now until migration completes): - Repositories in GitHub, pipelines in Azure DevOps for teams not yet moved, or in GitHub where their pipeline has been converted. - Metrics collected from both Azure DevOps and GitHub into the same Fabric model, with a source field so the origin is always visible.

Consolidated state (end state): - One GitHub Enterprise instance with one identity provider and single sign-on. - A central shared-workflows repository holding the build, test, and deployment patterns that Part A demonstrates. - Organization-level rulesets applied to all repositories, replacing per-repository rules. - Runner groups: hosted runners for standard products, and labelled self-hosted runners for legacy products. - Azure Boards kept in one Azure DevOps organization, with GitHub pull requests linked to work items. - Fabric as the single analytics layer, using the Bronze, Silver, and Gold pattern shown in Part B.

4. Operating model

- Platform team (small, dedicated): owns the shared workflows, rulesets, runner fleet, and onboarding. Treats the platform as an internal product with documented paths and a support queue.
- Product teams: own their repositories and their pipeline logic, and fix failures in their own code.
- Metric owner (one named person): owns metric definitions, approves changes to them, and signs off what is published to leaders.
- Security: owns the enforcement standard and the exception process.

The platform team's measure of success is adoption and reliability, not the number of repositories moved.

5. Migration waves

Repositories move only together with their pipelines. A repository whose pipeline is not ready stays where it is.

- Wave 1: standard products with simple pipelines and an active owner. Used to prove the shared workflow on real teams.
- Wave 2: standard products with complex pipelines or multiple stages. Requires the shared workflow catalogue to be extended first.
- Wave 3: legacy products. Moved only after a runner decision for each product: self-hosted x86 runner, or retirement.

Each wave is chosen from the pipeline inventory, not by repository count.

6. Governance

- Rulesets: applied first in audit mode (report failures, do not block), then required checks, then enforced on all repositories in each wave.
- Required check names are kept stable. Renaming a job breaks every merge, as seen in Part A, so changes to required check names need the platform team's approval.
- CODEOWNERS on workflow files, so a change to a shared workflow needs a platform reviewer.
- Exception register: any repository that cannot meet a rule gets a named owner, a reason, and an expiry date. Expired exceptions are escalated.
- Admin bypass is disabled, with a logged emergency path.

7. Trusted metrics

The metrics are useful only if the numbers are trusted. Part B showed the same source data can produce different lead times, depending on how commits are de-duplicated. The rules are therefore fixed first:

- One written definition for each DORA metric, owned by the metric owner.

- The commit-to-build link is the unit of truth, not the commit alone.
- A coverage figure is published with every lead-time number. Currently 14.9% of builds have a linked commit, so the figure is labelled partial until linkage reaches the agreed threshold of 80%.
- Data-quality checks run before publication. A failing error-level check stops the publish.
- Mean and median time to restore are both reported, with the median as the headline, because one outlier can dominate the mean.

No productivity or performance target is set from these metrics until linkage passes its threshold.

8. Legacy products

Legacy products that need x86 cannot use standard runners. For each product, the platform team and product owner choose one of two paths:

- Self-hosted Windows runner, labelled for x86, patched and owned, with a stated end date for review.
- Retirement or rewrite, with a funded plan.

Until one of these is chosen, the product stays on Azure DevOps. It is not forced onto a runner that cannot run it.

9. AI-enabled engineering

Proposed, not built. Each item needs human approval and measurable results before it is widened.

- Pull request summaries and risk notes, to help reviewers, not to approve changes.
- Failure triage: summarising failed build logs and grouping repeated failures.
- Automated dependency update pull requests, which a person merges.
- Agentic DevOps (an AI agent that proposes fixes) is deferred until the metrics are trusted, because an agent acting on disputed numbers multiplies the confusion.

10. Roadmap

First 90 days: - Days 0 to 30: freeze repository moves that have no pipeline plan. Inventory all 1,500 pipelines by product, type, and owner. Run a short developer friction survey to set a baseline. Name the metric owner. - Days 31 to 60: publish the shared workflow catalogue. Convert wave 1 pilot teams. Switch rulesets to audit mode. Design the x86 runner standard with Security. - Days 61 to 90: move wave 1 to required checks. Fix commit linkage at source for the largest product. Publish the first metric set with coverage labels. Decide the path for each legacy product.

Twelve months: - Quarter 1: stabilise and complete the inventory. - Quarter 2: convert wave 1 and wave 2 pipelines. Extend enforcement to those repositories. - Quarter 3:

move standard products in wave 3, bring metrics to the agreed linkage threshold, pilot one AI-assisted practice. - Quarter 4: complete legacy decisions, retire the Azure DevOps pipelines that are no longer needed, and review the end state.

Each milestone has a measure. Estimates of effort and cost are to be confirmed with the platform team and are not stated here.

11. Risks and responses

- Pipelines stay behind: the wave rule stops repository moves, and the inventory shows the backlog.
- Enforcement blocks teams: audit mode first, and a stable check name policy.
- Legacy products stall: a named decision per product, with an owner and date.
- Metrics stay disputed: one owner, one definition, and publish rules that show coverage.
- Friction rises: a friction measure in the baseline survey, repeated each quarter.

12. Decisions required

- CTO: approve the re-scope, name the metric owner, and set the end-state date.
- Security: approve the phased enforcement standard and the self-hosted runner standard for legacy products.
- Product leadership: accept a revised pace, agree which products go first, and agree how developer friction is measured.

13. Evidence from Parts A and B

- Part A: the converted pipeline, the shared build workflow, pinned actions, required checks, and the documented migration mapping.
- Part B: the Bronze, Silver, and Gold pipeline, the data-quality checks, the linkage finding (14.9%), and the mean and median restore figures (2.03 and 43.64 hours).

14. Limitations

- Evidence comes from one pipeline, one repository, and one synthetic dataset.
- The metric figures use 11 incidents and cannot support firm conclusions about reliability.
- Cost and effort estimates are not included and must be produced with the platform team.